



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение  
высшего образования

«МИРЭА – Российский технологический университет»

**РТУ МИРЭА**

**Институт информационных технологий (ИТ)**

**Кафедра инструментального и прикладного программного обеспечения (ИиППО)**

**КУРСОВАЯ РАБОТА**

по дисциплине: Архитектура клиент-серверных приложений

по профилю: Разработка программных продуктов и проектирование информационных систем

направления профессиональной подготовки: 09.03.04 «Программная инженерия»

Тема: «Разработка архитектуры системы управления онлайн-обучением с поддержкой тестирования и оценки знаний»

Студент: Миркин Кирилл Леонидович

Группа: ИКБО-10-23

Работа представлена к защите \_\_\_\_\_ (дата)

\_\_\_\_\_/Миркин К.Л./  
(подпись и ф.и.о. студента)

Руководитель: профессор, Мифтахов Эльдар Наилевич

Работа допущена к защите \_\_\_\_\_ (дата)

\_\_\_\_\_/Мифтахов Э.Н./  
(подпись и ф.и.о. рук-ля)

Оценка по итогам защиты: \_\_\_\_\_

\_\_\_\_\_/ Мифтахов Эльдар Наилевич, профессор, д.ф.-м.н. \_\_\_\_\_/

\_\_\_\_\_/ Ермаков Сергей Романович, старший преподаватель \_\_\_\_\_/

(подписи, дата, ф.и.о., должность, звание, уч. степень двух преподавателей, принявших защиту)

2025 г.

Заведующему кафедрой  
инструментального и прикладного  
программного обеспечения (ИиППО)  
Института информационных технологий (ИТ)  
Болбакову Роману Геннадьевичу

От студента Миркина

Кирилла Леонидовича

ФИО

ИКБО-10-23

группа

3 курс

курс

Контакт: 8(916) 485-67-25, mirkin.k.l@edu.mirea.ru

### Заявление

Прошу утвердить мне тему курсовой работы по дисциплине «Архитектура клиент-серверных приложений» образовательной программы бакалавриата 09.03.04 (Программная инженерия). Тема: Разработка архитектуры системы управления онлайн-обучением с поддержкой тестирования и оценки знаний.

Приложение: лист задания на КР/КП в 2-ух экземплярах на двухстороннем листе (проект)

Подпись студента

/Миркин К.Л./

подпись

ФИО

Дата

Подпись руководителя

/ проф. Мифтахов Э.Н./

подпись

Должность, ФИО

Дата

Согласовано \_\_\_\_\_ /к.т.н., зав. кафедрой ИиППО, Болбаков Р.Г.



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение  
высшего образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)  
Кафедра инструментального и прикладного программного обеспечения (ИиППО)

## ЗАДАНИЕ на выполнение курсовой работы

по дисциплине: Архитектура клиент-серверных приложений

по профилю: Разработка программных продуктов и проектирование информационных систем  
направления профессиональной подготовки: Программная инженерия (09.03.04)

Студент: Миркин Кирилл Леонидович

Группа: ИКБО-10-23

Срок представления к защите: 08.12.2025

Руководитель: д.ф.-м.н., профессор, Мифтахов Эльдар Наилевич

**Тема:** «Разработка архитектуры системы управления онлайн-обучением с поддержкой тестирования и оценки знаний»

**Исходные данные:** Архитектурные стили и паттерны: клиент-серверная архитектура, многослойная архитектура, API-first, Feature-Sliced Design, Reverse Proxy; языки моделирования и нотации: UML, DFD, ER; используемые технологии: языки программирования – Python TypeScript; фреймворки – FastAPI, React; СУБД – PostgreSQL; инструменты: SQLAlchemy, Vite; открытое ПО: Visual Studio Code, Draw.io, Git, Docker.

**Перечень вопросов, подлежащих разработке, и обязательного графического материала:** 1. Дать описание предметной области, для которой разрабатывается ПО; 2. Определить и сформировать перечень функциональных и нефункциональных требований к ПО; 3. Определить и описать ограничения проекта; 4. Провести обзор архитектурных стилей для выбора наиболее подходящей архитектуры; 5. Разработать набор детализированных диаграмм архитектуры; 6. Изучить и описать популярные паттерны проектирования, которые можно применить для данного проекта; 7. Обосновать выбор технологий для реализации выбранной архитектуры; 8. Разработать прототип приложения с минимально жизнеспособным набором функций (MVP) и развернуть его в соответствующей инфраструктуре.

Руководителем произведён инструктаж по технике безопасности, противопожарной технике и правилам внутреннего распорядка.

Зав. кафедрой ИиППО: \_\_\_\_\_ / Болбаков Р. Г. /, «\_\_\_\_» \_\_\_\_\_ 2025 г.

Задание на КР выдал: \_\_\_\_\_ / Мифтахов Э.Н. /, «\_\_\_\_» \_\_\_\_\_ 2025 г.

Задание на КР получил: \_\_\_\_\_ / Миркин К.Л. /, «\_\_\_\_» \_\_\_\_\_ 2025 г.

## РЕФЕРАТ

Отчёт 45 с., 9 рис., 5 табл., 26 ист.

### CLEAN ARCHITECTURE, FASTAPI, REST API, СЕРВЕРНАЯ ЧАСТЬ

Миркин К.Л., Курсовая работа направления подготовки «Программная инженерия» предмета «Бэкенд-разработка» на тему «Серверная часть веб-приложения для управления образовательными курсами»: М. 2026 г., МИРЭА – Российский технологический университет (РТУ МИРЭА), Институт информационных технологий (ИИТ), кафедра инструментального и прикладного программного обеспечения (ИиППО)

Курсовая работа посвящена разработке серверной части веб-приложения для управления образовательными курсами. Выполнен анализ предметной области, сформированы функциональные и нефункциональные требования к системе управления обучением (LMS), проведён обзор существующих решений с построением матрицы отслеживаемости аналогов.

Проведён анализ архитектурных паттернов DDD, Clean Architecture и MVC, выполнен их сравнительный анализ и обоснован выбор Clean Architecture в качестве основного подхода к проектированию серверной части. Разработана детальная архитектура системы на основе Clean Architecture с описанием слоёв API, сервисов, репозиторий и моделей.

Реализована серверная часть на языке Python с использованием фреймворка FastAPI, СУБД PostgreSQL и библиотеки SQLAlchemy. Приведены листинги кода, описана структура REST API, слой базы данных и механизмы аутентификации. Корректность разработанной архитектуры подтверждена реализованным прототипом системы.

## СОДЕРЖАНИЕ

|                                                              |    |
|--------------------------------------------------------------|----|
| Определения, обозначения и сокращения .....                  | 7  |
| Введение .....                                               | 9  |
| 1 Анализ предметной области и требований .....               | 11 |
| 1.1 Описание предметной области .....                        | 11 |
| 1.2 Требования к системе .....                               | 11 |
| 1.2.1 Функциональные требования .....                        | 11 |
| 1.2.2 Нефункциональные требования .....                      | 12 |
| 1.2.3 Ограничения проекта .....                              | 13 |
| 1.3 Анализ существующих LMS-решений .....                    | 13 |
| 1.4 Матрица отслеживаемости аналогов .....                   | 15 |
| 1.5 Итоги анализа предметной области .....                   | 16 |
| 2 Выбор и обоснование технологий .....                       | 17 |
| 2.1 Анализ архитектурных паттернов .....                     | 17 |
| 2.1.1 Domain-Driven Design .....                             | 17 |
| 2.1.2 Clean Architecture .....                               | 17 |
| 2.1.3 MVC и MVT в контексте серверной разработки .....       | 18 |
| 2.2 Сравнение паттернов и обоснование выбора .....           | 19 |
| 2.3 Обоснование технологического стека .....                 | 20 |
| 2.4 Итоги выбора технологий .....                            | 22 |
| 3 Разработка архитектуры приложения .....                    | 23 |
| 3.1 Слои архитектуры .....                                   | 23 |
| 3.2 Dependency Injection как механизм связывания слоёв ..... | 24 |
| 3.3 Обработка ошибок .....                                   | 24 |
| 3.4 Диаграмма компонентов .....                              | 25 |
| 3.5 Проектирование базы данных .....                         | 27 |
| 3.6 Результаты проектирования .....                          | 29 |
| 4 Разработка серверной части интернет-ресурса .....          | 30 |
| 4.1 Реализация REST API на FastAPI .....                     | 30 |
| 4.2 Реализация сервисного слоя .....                         | 31 |
| 4.3 Слой безопасности .....                                  | 33 |
| 4.4 Управление конфигурацией и запуск .....                  | 34 |

|                                        |    |
|----------------------------------------|----|
| 4.5 Тестирование .....                 | 35 |
| 4.6 Клиентское представление .....     | 36 |
| 4.7 Итоги разработки .....             | 40 |
| Заключение .....                       | 41 |
| Список использованных источников ..... | 43 |

## ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящем отчёте применяют следующие определения, обозначения и сокращения.

|                                            |                                                                                                                                                              |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| API (Application Programming Interface)    | – программный интерфейс, определяющий способы взаимодействия компонентов программного обеспечения.                                                           |
| Argon2                                     | – алгоритм хеширования паролей, устойчивый к атакам с использованием GPU и ASIC.                                                                             |
| Clean Architecture                         | – архитектурный паттерн, предложенный Робертом Мартином, обеспечивающий независимость бизнес-логики от фреймворков, баз данных и внешних интерфейсов.        |
| DDD (Domain-Driven Design)                 | – подход к проектированию программного обеспечения, ориентированный на моделирование предметной области и использование единого языка (Ubiquitous Language). |
| Dependency Injection (DI)                  | – паттерн проектирования, при котором зависимости компонентов передаются извне, а не создаются внутри компонента.                                            |
| DTO (Data Transfer Object)                 | – объект передачи данных, используемый для обмена структурированной информацией между слоями приложения.                                                     |
| ER-диаграмма (Entity-Relationship Diagram) | – диаграмма «сущность–связь», используемая для моделирования структуры данных и отношений между сущностями.                                                  |
| FastAPI                                    | – современный веб-фреймворк для Python, ориентированный на создание REST API с поддержкой асинхронности, автоматической валидации данных и документирования. |
| HTTP (HyperText Transfer Protocol)         | – протокол передачи данных между клиентом и сервером в веб-среде.                                                                                            |

|                                        |                                                                                                                                                  |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| JWT (JSON Web Token)                   | – компактный и самодостаточный формат для передачи данных в виде JSON-объекта, используемый для аутентификации.                                  |
| LMS (Learning Management System)       | – система управления обучением, предназначенная для организации образовательного процесса, хранения материалов и контроля результатов.           |
| MVC (Model-View-Controller)            | – архитектурный паттерн, разделяющий приложение на три взаимосвязанных компонента: модель, представление и контроллер.                           |
| ORM (Object-Relational Mapping)        | – технология отображения объектных структур в реляционные базы данных.                                                                           |
| Repository                             | – паттерн, инкапсулирующий логику доступа к данным и предоставляющий интерфейс для операций с сущностями.                                        |
| REST (Representational State Transfer) | – архитектурный стиль взаимодействия распределённых систем, использующий принципы унифицированного интерфейса и обмена представлениями ресурсов. |
| Service Layer                          | – слой бизнес-логики приложения, обрабатывающий прикладные операции и обращающийся к репозиториям.                                               |
| SQL (Structured Query Language)        | – язык запросов, используемый для управления реляционными базами данных.                                                                         |
| SQLAlchemy                             | – библиотека ORM и инструмент работы с реляционными базами данных в Python.                                                                      |
| СУБД                                   | – система управления базами данных.                                                                                                              |
| Фреймворк                              | – программный каркас, упрощающий разработку приложений и задающий архитектурные принципы.                                                        |



## ВВЕДЕНИЕ

Разработка серверной части современных веб-приложений представляет собой одну из ключевых дисциплин в области программной инженерии. Серверная часть (backend) отвечает за обработку бизнес-логики, хранение и предоставление данных, аутентификацию пользователей, обеспечение безопасности и масштабируемости системы. Качество архитектурных решений, принятых на этапе проектирования серверной части, напрямую определяет такие характеристики итогового продукта, как производительность, надёжность, сопровождаемость и возможность дальнейшего расширения функциональности [1].

Одной из актуальных областей применения backend-разработки являются системы управления обучением (Learning Management Systems, LMS) [2]. Такие системы обеспечивают организацию учебного процесса в цифровой среде: управление курсами и учебными материалами, проведение тестирования, отслеживание успеваемости и коммуникацию между участниками образовательного процесса. С ростом популярности дистанционного и смешанного форматов обучения возрастают требования к архитектуре подобных систем, что делает задачу проектирования и реализации их серверной части особенно значимой.

Актуальность выбранной темы определяется необходимостью разработки серверной части современной LMS-системы с применением актуальных архитектурных паттернов и технологического стека. Особое внимание уделяется Clean Architecture – подходу, обеспечивающему независимость бизнес-логики от фреймворков, баз данных и внешних интерфейсов. Применение данного паттерна позволяет создать систему, устойчивую к изменениям требований и удобную для сопровождения [3].

Цель работы – разработка серверной части веб-приложения для управления образовательными курсами на основе архитектурного паттерна Clean Architecture с использованием современных технологий и инструментов.

Для достижения поставленной цели необходимо решить следующие задачи:

- выполнить анализ предметной области и определить требования к серверной части LMS-системы;
- провести обзор и сравнительный анализ архитектурных паттернов DDD, Clean Architecture и MVC, применимых к разработке серверной части;
- обосновать выбор технологического стека (FastAPI, PostgreSQL, SQLAlchemy) для реализации серверной части;
- разработать архитектуру серверной части на основе Clean Architecture с детальным описанием слоёв;
- реализовать серверную часть приложения, включая REST API, слой бизнес-логики и слой доступа к данным;
- привести листинги кода, демонстрирующие применение выбранных архитектурных решений.

Объект исследования – процессы организации и сопровождения онлайн-обучения.

Предмет исследования – архитектурные и технологические решения при разработке серверной части системы управления обучением с применением Clean Architecture.

## **1 Анализ предметной области и требований**

### **1.1 Описание предметной области**

Система управления онлайн-обучением (LMS, Learning Management System) – программная платформа, предназначенная для организации, сопровождения и контроля учебного процесса в цифровой среде. Платформа обеспечивает студентам доступ к учебным курсам, материалам и заданиям, а преподавателям – инструменты управления контентом, формирования и проверки заданий, оценки результатов.

Предметная область охватывает несколько ключевых компонентов:

- **учебные курсы** с теоретическими материалами, практическими заданиями и тестами;
- **пользователей** с разграниченными ролями: преподаватели и студенты;
- **механизмы тестирования и оценки знаний**, обеспечивающие контроль усвоения материала;
- **хранилище данных** – профили пользователей, курсы, результаты, учебные артефакты.

Типичные сценарии использования LMS включают регистрацию и аутентификацию пользователей, управление курсами и загрузку обучающих материалов, формирование и выдачу заданий, сохранение результатов и формирование отчётности. Структура и функциональные возможности современных LMS рассмотрены в исследовании Rosário и Dias [2]. Принципы программной инженерии, применяемые при построении подобных систем – модульность, разделение ответственности, изоляция слоёв, – описаны в учебных пособиях по дисциплине [4].

### **1.2 Требования к системе**

#### **1.2.1 Функциональные требования**

На основе анализа предметной области сформированы функциональные требования к серверной части, отражающие минимально необходимый объём возможностей системы.

Таблица 1 – Функциональные требования к серверной части системы

| Группа                                 | Содержание требования                                                                                                                                   |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Управление курсами и уроками           | Преподаватель создаёт учебные классы, формирует уроки и публикует их. Студент получает доступ к назначенным урокам.                                     |
| Пользователи и роли                    | Регистрация, аутентификация и разграничение прав. Поддержка ролей «преподаватель» и «студент» с возможностью переключения активной роли.                |
| База задач и домашние задания          | Преподаватель ведёт независимую базу задач, используемую в разных домашних заданиях. Домашнее задание формируется как набор задач, привязанный к уроку. |
| Тестирование и автоматическая проверка | Студент отправляет ответы; система автоматически сверяет их с эталоном, фиксирует результат, число попыток и затраченное время.                         |
| Прогресс и статистика                  | Студент видит личный прогресс; преподаватель – агрегированную статистику по классу.                                                                     |
| Чат класса                             | Обмен сообщениями в реальном времени между участниками класса с сохранением истории.                                                                    |
| Приглашения в классы                   | Преподаватель генерирует код-приглашение; студент вступает в класс по коду.                                                                             |
| Теоретические материалы                | Иерархические теоретические материалы, привязанные к урокам и доступные студентам после публикации.                                                     |

### 1.2.2 Нефункциональные требования

Нефункциональные требования определяют эксплуатационные характеристики системы и влияют на архитектурные решения [5]. Оформление проектной документации ведётся по ГОСТ 7.32-2017 [6]. Система ориентирована на городскую образовательную среду с числом пользователей до 100 000 человек.

Таблица 2 – Нефункциональные требования к системе

| Критерий           | Метрика                     | Значение                      | Обоснование                                                                           |
|--------------------|-----------------------------|-------------------------------|---------------------------------------------------------------------------------------|
| Производительность | Время отклика               | 0–1000 мс                     | Отклик до 100 мс воспринимается как мгновенный; до 1000 мс – как естественный.        |
|                    | Время загрузки интерфейса   | $\leq 3$ с                    | Обеспечивает комфортную навигацию между уроками и заданиями.                          |
| Нагрузка           | Пиковая активность          | 5 000–10 000 пользователей    | При 100 000 зарегистрированных пользователей пиковая активность – 10% в учебные часы. |
|                    | Интенсивность запросов, RPS | 500–1000 RPS                  | $10 \sim 000 \times (6/60) \approx 1000$ RPS при 6 вызовах API на одно действие.      |
| Надёжность         | Доступность                 | $\geq 99,5$ %                 | Непрерывная доступность в учебные часы и периоды контрольных мероприятий.             |
| Совместимость      | Браузеры                    | Chrome, Firefox, Edge, Safari | Поддержка актуальных версий основных браузеров в школьной и домашней среде.           |

### 1.2.3 Ограничения проекта

Система разрабатывается одним исполнителем в учебные сроки, что ограничивает применение сложных инфраструктурных решений и предопределяет необходимость ориентироваться на ключевые сценарии использования. Архитектура должна оставаться компактной и допускать дальнейшее расширение без переписывания ядра.

### 1.3 Анализ существующих LMS-решений

Для обоснования требований и выявления конкурентных преимуществ разрабатываемой системы рассмотрены четыре широко используемые платформы: Moodle, Google Classroom, Stepik и Фоксфорд [7–10]. Цель анализа – понять, насколько хорошо каждая из них закрывает сформулированные требования, и определить, в каких аспектах разрабатываемое решение может их превзойти.

**Moodle** – наиболее распространённая LMS с открытым исходным кодом (более 20 лет на рынке) [7]. Написана на PHP и развёртывается

на собственном сервере. Сильная сторона Moodle – богатство плагинов и зрелый механизм тестирования, поддерживающий десятки типов вопросов. Слабые стороны: сложность развёртывания и поддержки, устаревший пользовательский интерфейс и отсутствие удобного современного REST API, что затрудняет создание сторонних клиентов и интеграций. Независимая база задач в Moodle реализована лишь частично: вопросы хранятся в категориях, но привязаны к отдельным курсам, а не к платформе глобально.

**Google Classroom** – облачная платформа Google, интегрированная с Google Workspace for Education [8]. Ориентирована на максимальную простоту: курс создаётся за несколько минут без установки инфраструктуры. Платформа хорошо подходит для выдачи заданий в формате документов и ссылок на Google Drive, но практически не поддерживает автоматическую проверку ответов – задания проверяются вручную. REST API существует, однако покрывает лишь базовые операции с курсами и заданиями; WebSocket-чат и приглашения по коду отсутствуют. Развернуть платформу на собственном сервере невозможно.

**Stepik** – российская платформа с сильным акцентом на интерактивное тестирование [9]. Stepik поддерживает разнообразные типы задач (строковые, числовые, код на разных языках, выбор из вариантов) с полностью автоматической проверкой – это её ключевое преимущество перед остальными аналогами. Платформа предоставляет REST API для курсов и задач. Основное ограничение: Stepik ориентирован на публикацию открытых или платных курсов, а не на модель «закрытый класс с преподавателем», которая нужна для разрабатываемой системы. Создание учебных классов, управление домашними заданиями и realtime-чат не являются нативными функциями платформы.

**Фоксфорд** – российская онлайн-школа для учеников 1–11 классов [10]. По функциональной модели Фоксфорд наиболее близок к разрабатываемой системе: есть классы, уроки, домашние задания, чат и статистика успеваемости. Однако это полностью закрытая коммерческая система без публичного API и возможности развернуть у себя. Исходный код закрыт, интеграция с внешними инструментами не предусмотрена.

## 1.4 Матрица отслеживаемости аналогов

Анализ аналогов позволяет сопоставить функциональность платформ и понять, насколько каждая из них соответствует сформулированным требованиям. Для наглядности результаты анализа разделены на две таблицы: функциональные возможности представлены в таблице 3, а архитектурные характеристики сопоставлены в таблице 4.

Таблица 3 – Матрица функциональных возможностей LMS-аналогов

| Функция         | Moodle                                     | Google Classroom                           | Stepik                                   | Фоксфорд                                   |
|-----------------|--------------------------------------------|--------------------------------------------|------------------------------------------|--------------------------------------------|
| Курсы и уроки   | Разделы, страницы, медиа, условный доступ. | Темы с потоком заданий, без чётких уроков. | Уроки, видео, тесты, дедлайны.           | Структурированные уроки с материалами.     |
| База задач и ДЗ | Банк вопросов привязан к курсу.            | Задания в виде файлов/ссылок.              | Глобальная база задач для разных курсов. | ДЗ без выделения задач в сущность.         |
| Автопроверка    | Quiz-модуль, 15 типов вопросов.            | Ручная проверка преподавателем.            | Мощный автогрейдер, проверка кода.       | Тесты; свободный ввод проверяется вручную. |
| Статистика      | Отчёты по оценкам, журнал доступа.         | Список сданных работ, оценки.              | Баллы, рейтинги, аналитика по шагам.     | Оценки, посещаемость, прогресс по урокам.  |
| Чат (realtime)  | Асинхронные форумы и сообщения.            | Внешний Google Chat.                       | Асинхронные комментарии к задачам.       | Встроенный чат класса в реальном времени.  |
| Приглашения     | Ключи записи на курс.                      | Код класса для вступления.                 | Запись через поиск или ссылку.           | Запись управляется администратором.        |

В таблице 4 приведено сопоставление платформ по инфраструктурным параметрам.

Таблица 4 – Матрица архитектурных характеристик LMS-аналогов

| Характеристика | Moodle                                 | Google Classroom                 | Stepik                                  | Фоксфорд                   |
|----------------|----------------------------------------|----------------------------------|-----------------------------------------|----------------------------|
| Открытый код   | Да (GPL-3.0).                          | Нет.                             | Нет.                                    | Нет.                       |
| Развёртывание  | Собственный сервер (PHP, БД, плагины). | Облако Google.                   | Облако Stepik.                          | Закрытая SaaS-система.     |
| REST API       | Сложный и устаревший.                  | Ограниченный (базовые операции). | Полноценный (открыт для разработчиков). | Публичный API отсутствует. |

Матрицы показывают, что у каждой платформы есть свои ниши: Moodle силён в тестировании, Stepik – в автопроверке задач, Google Classroom – в простоте. При этом ни одна из систем не сочетает одновременно собственное развёртывание, открытый код, realtime-чат и независимую переиспользуемую базу задач с REST API. Это определяет нишу разрабатываемой системы.

### 1.5 Итоги анализа предметной области

Выполнен анализ предметной области систем управления обучением. Сформированы функциональные и нефункциональные требования к серверной части, определены ограничения проекта. Рассмотрены четыре LMS-платформы – Moodle, Google Classroom, Stepik, Фоксфорд – и построена матрица отслеживаемости. Анализ показал, что ни одна из рассматриваемых платформ не покрывает одновременно все ключевые требования проекта. Полученные результаты формируют основу для выбора технологий и проектирования архитектуры.



## **2 Выбор и обоснование технологий**

### **2.1 Анализ архитектурных паттернов**

Архитектурный паттерн определяет способ организации кода, распределение ответственности и характер взаимодействия компонентов. От его выбора зависят тестируемость, сопровождаемость и расширяемость системы [1]. В данной главе рассматриваются три паттерна, применяемых при разработке серверной части: Domain-Driven Design (DDD), Clean Architecture и Model-View-Controller (MVC).

#### **2.1.1 Domain-Driven Design**

Domain-Driven Design – подход, предложенный Эриком Эвансом, в котором ключевой единицей является модель предметной области [11]. Основные концепции:

- **Сущности (Entities)** – объекты с уникальным идентификатором и изменяемым состоянием.
- **Объекты-значения (Value Objects)** – неизменяемые объекты, описываемые атрибутами.
- **Агрегаты** – совокупности сущностей, управляемые как единое целое.
- **Репозитории** – механизмы получения и хранения агрегатов.
- **Доменные сервисы** – операции, не принадлежащие ни одной сущности.

DDD обеспечивает точное соответствие программной модели бизнес-требованиям, однако требует активного участия экспертов предметной области и значительных трудозатрат на начальное моделирование. В типичном учебном проекте с одним исполнителем DDD избыточен: полноценная доменная модель оправдывает себя при сложных, постоянно меняющихся бизнес-правилах.

#### **2.1.2 Clean Architecture**

Clean Architecture, предложенная Робертом Мартином [3], организует код в виде концентрических слоёв. Зависимости направлены только внутрь: внешние слои могут зависеть от внутренних, но не наоборот. Ключевые принципы:

- **Независимость от фреймворков** – бизнес-логика не зависит от конкретного веб-фреймворка.

– **Тестируемость** – бизнес-правила проверяются без поднятия HTTP-сервера или базы данных.

– **Независимость от БД** – замена СУБД не затрагивает бизнес-логику.

В серверной части выделяются четыре слоя:

1) **API Layer** – приём запросов, валидация, маршрутизация.

2) **Service Layer** – реализация бизнес-сценариев.

3) **Repository Layer** – абстракция доступа к данным.

4) **Models Layer** – ORM-модели, конфигурация БД, внешние интеграции.

Паттерн хорошо сочетается с REST-архитектурой: API-контроллеры образуют внешний слой, а бизнес-логика остаётся независимой от протокола.

### **2.1.3 MVC и MVT в контексте серверной разработки**

Model-View-Controller – один из наиболее известных архитектурных паттернов, исторически применяемых в веб-разработке. Его основная идея состоит в разделении приложения на три логических части: модель, отвечающую за данные и связанные с ними правила; представление, отвечающее за отображение информации; и контроллер, принимающий запрос пользователя и координирующий дальнейшую работу системы. Преимущество MVC заключается в простоте понимания и относительно низком пороге входа, особенно для приложений с выраженным CRUD-характером.

Однако в серверной разработке на Python классический MVC в чистом виде встречается редко. Наиболее распространённый пример – фреймворк Django, который использует близкую, но не тождественную модель MVT (Model-View-Template). В MVT слой Template отвечает за отображение данных, а слой View фактически совмещает функции контроллера и части прикладной логики. Это означает, что серверный код в приложениях на Django по своей природе сильнее связан с самим фреймворком, чем в системах, построенных на FastAPI или Flask.

С практической точки зрения различие между MVC и MVT важно не как формальная терминологическая деталь, а как характеристика степени зависимости прикладного кода от инфраструктуры. В MVC/MVT обработчики запросов обычно тесно связаны с жизненным циклом HTTP-

запроса, объектами фреймворка и механизмами ORM. При небольшом объёме функциональности это допустимо, однако по мере роста проекта затрудняет изоляцию бизнес-логики, повторное использование кода и построение независимого слоя тестирования. Именно поэтому MVC и MVT удобны как стартовая модель для небольших веб-приложений, но хуже подходят для систем, где требуется длительное развитие серверной части и явное разделение ответственности.

## **2.2 Сравнение паттернов и обоснование выбора**

Для обоснованного выбора паттерна необходимо рассмотреть, как каждый из трёх подходов проявляет себя в контексте разрабатываемой системы. Ниже дано сравнение по критериям, существенным для backend-проекта с одним разработчиком, REST API и требованием тестируемости. Итоговое сопоставление приведено в таблице 5.

**Тестируемость** – важнейший критерий для проекта, где качество архитектуры надо подтверждать автоматически. Clean Architecture и DDD обеспечивают высокую тестируемость: сервисы и доменные объекты не зависят от фреймворка, их можно тестировать без поднятия HTTP-сервера. В MVC и MVT контроллеры тесно связаны с фреймворком, а бизнес-логика нередко попадает прямо в них – это существенно усложняет написание изолированных тестов.

**Независимость от инфраструктуры** – ключевое требование для систем, где технологический стек может меняться. Clean Architecture обеспечивает полную независимость бизнес-логики от фреймворка и СУБД через явные слои. DDD достигает схожего результата, но через агрегаты и доменные события. MVC/MVT жёстко привязан к фреймворку: замена Django на FastAPI потребует переписывания значительной части кода.

**Применимость в контексте задачи** – DDD оправдан при сложной предметной области с десятками агрегатов и постоянно меняющимися бизнес-правилами. В данном проекте предметная область достаточно прозрачна (классы, уроки, задания, результаты), поэтому полноценная DDD-модель избыточна. Clean Architecture даёт чёткую структуру без лишней сложности. MVC/MVT – наиболее простой вход, но у него нет встроенного механизма

для изоляции бизнес-логики: при росте системы она неизбежно начинает «протекать» в контроллеры.

Таблица 5 – Сравнение архитектурных паттернов по ключевым критериям

| Критерий                    | DDD                                                     | Clean Architecture                                          | MVC / MVT                                                              |
|-----------------------------|---------------------------------------------------------|-------------------------------------------------------------|------------------------------------------------------------------------|
| Тестируемость бизнес-логики | Высокая: доменные объекты независимы.                   | Высокая: сервисный слой изолирован от фреймворка.           | Низкая: логика смешана с контроллерами.                                |
| Независимость от фреймворка | Средняя: достигается через агрегаты, но требует усилий. | Полная: слои разделены явно, зависимости направлены внутрь. | Низкая: контроллер привязан к фреймворку (Django views, Flask routes). |
| Сложность внедрения         | Высокая: агрегаты, события, ubiquitous language.        | Средняя: четыре слоя, правило зависимостей.                 | Низкая: структуру задаёт сам фреймворк.                                |
| Применимость в проекте      | Избыточна: предметная область проекта невелика.         | Оптимальна: баланс структуры и сложности.                   | Упрощённо: подходит для простых CRUD-приложений, не для роста.         |

Таблица 5 подтверждает выбор Clean Architecture: при средней сложности внедрения она даёт полную независимость бизнес-логики и высокую тестируемость, что соответствует требованиям проекта.

### 2.3 Обоснование технологического стека

**Python** является одним из наиболее популярных языков для серверной веб-разработки благодаря читаемому синтаксису, богатой экосистеме и поддержке асинхронного программирования через `asyncio` [12]. Система аннотаций типов (type hints) обеспечивает статическую проверку кода без отказа от динамической природы языка [13]. В проекте используется версия 3.12+, что обусловлено не только актуальностью данной ветки языка, но и рядом практических преимуществ по сравнению с Python 3.11.

Во-первых, Python 3.12 продолжает курс на повышение производительности интерпретатора, начатый в 3.11. Для серверного приложения это означает уменьшение накладных расходов на обработку типовых операций: создание объектов, вызовы функций, работу с коллекциями и строками. Хотя прирост производительности не является единственным

критерием выбора языка, в backend-разработке он важен, поскольку такие операции выполняются при каждом HTTP-запросе.

Во-вторых, версия 3.12 улучшает работу системы типов и делает код более удобным для статического анализа. Для проекта, в котором активно используются Pydantic, SQLAlchemy, FastAPI и т.п., это особенно значимо: чем точнее интерпретатор и инструменты анализа понимают аннотации, тем надёжнее проверяется согласованность между схемами данных, сервисами и маршрутизаторами. В результате уменьшается вероятность ошибок на стыке слоёв архитектуры. Дополнительным практическим преимуществом является поддержка более современного синтаксиса обобщённых типов и общее повышение качества работы инструментов типизации в актуальном стеке библиотек.

В-третьих, Python 3.12 представляет собой уже устоявшуюся и широко поддерживаемую версию языка для современного веб-стека. Использование именно этой ветки позволяет опираться на актуальные версии FastAPI, Pydantic v2 и SQLAlchemy 2.0 без необходимости учитывать ограничения более ранних интерпретаторов. По сравнению с Python 3.11 различие здесь состоит не столько в наличии одной «критической» функции, сколько в более предсказуемой и зрелой среде разработки, лучше согласованной с актуальными библиотеками проекта.

**FastAPI** – современный Python-фреймворк для создания REST API, один из наиболее востребованных по данным опроса Stack Overflow 2025 [14–16]. Как показано на рисунке 1, популярность FastAPI стремительно растёт благодаря его высокой производительности и удобству разработчика. Выбор FastAPI обоснован следующим: фреймворк не диктует архитектуру, обеспечивает полноценный asyncio, встроенный Dependency Injection и автоматическую генерацию OpenAPI-документации.

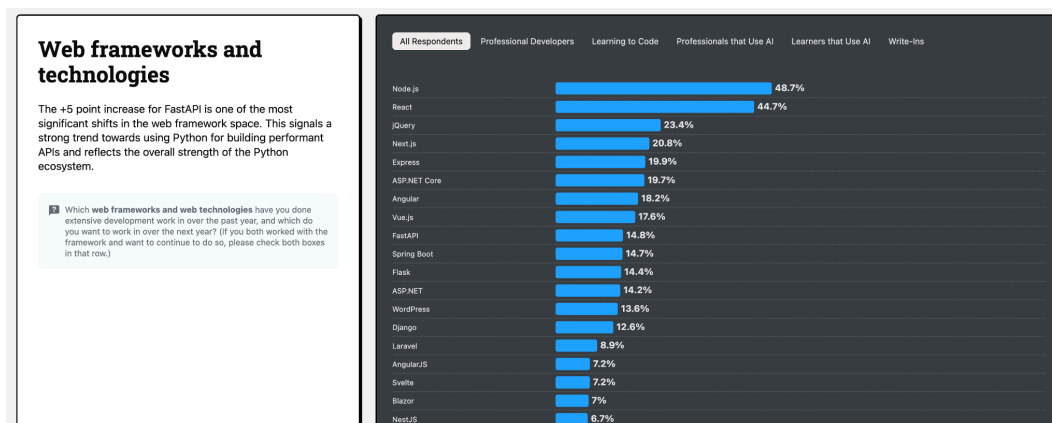


Рисунок 1 – Сравнение популярности веб-фреймворков по данным Stack Overflow

**PostgreSQL** – реляционная СУБД с поддержкой ACID-транзакций, расширенных типов данных и развитой системы индексов [17,18]. PostgreSQL является отраслевым стандартом для веб-приложений среднего и крупного масштаба.

**SQLAlchemy 2.0 (asyncio)** – зрелая ORM для Python с поддержкой асинхронного драйвера asyncpg [19]. Реализует паттерн Data Mapper, не привязанный к конкретному фреймворку, что соответствует принципам Clean Architecture [5]. Миграции управляются через Alembic [20].

**React 18 + TypeScript** – стек клиентской части [21]. Строгая типизация TypeScript обеспечивает предсказуемость кода; архитектура клиента построена на Feature-Sliced Design [22], что согласуется с разделением ответственности, принятым в Clean Architecture.

## 2.4 Итоги выбора технологий

Рассмотрены три архитектурных паттерна – DDD, Clean Architecture, MVC/MVT. Показана связь между выбором паттерна и Python-фреймворком: Django жёстко задаёт MVT, тогда как FastAPI не ограничивает архитектуру. По результатам сравнения, приведённого в таблице 5, выбрана Clean Architecture как обеспечивающая оптимальное соотношение тестируемости, независимости от инфраструктуры и практической сложности. Определён технологический стек: Python, FastAPI, PostgreSQL, SQLAlchemy для серверной части и React + TypeScript для клиентской.

## 3 Разработка архитектуры приложения

### 3.1 Слои архитектуры

На основе выбранного паттерна спроектирована архитектура серверной части. Код организован в четыре слоя с однонаправленными зависимостями: от внешних к внутренним [3,23].

- **API Layer** – внешний слой: приём HTTP-запросов, валидация через Pydantic, маршрутизация, формирование ответов. Не содержит бизнес-логики – делегирует вызовы сервисному слою.

- **Service Layer** – слой бизнес-логики: реализация сценариев использования. Сервисы не зависят от HTTP-контекста, используют репозитории для доступа к данным и могут тестироваться изолированно.

- **Repository Layer** – слой доступа к данным: паттерн Repository скрывает детали SQLAlchemy ORM от сервисного слоя.

- **Models Layer** – инфраструктурный слой: ORM-модели, конфигурация базы данных, миграции, внешние интеграции (Redis для realtime-чата).

Дополнительно выделены вспомогательные пакеты: **schemas** (Pydantic DTO для валидации входных данных и формирования ответов), **core** (конфигурация, JWT, хеширование), **realtime** (WebSocket-подсистема). Все они находятся вне основных слоёв и не нарушают правило зависимостей.

Структура директорий непосредственно отражает разбивку на слои:

```
app/
├─ api/           # API Layer: роутеры, DI, ошибки, middleware
│  ├─ v1/         #   эндпоинты по группам (auth, classrooms, ...)
│  ├─ http_errors.py #   фабрики HTTPException
│  └─ errors.py    #   глобальные обработчики
├─ services/      # Service Layer: бизнес-сценарии
├─ repositories/  # Repository Layer: SQL-запросы через ORM
├─ models/        # Models Layer: ORM-модели SQLAlchemy
├─ schemas/       # Pydantic DTO
├─ core/          # конфигурация, безопасность, утилиты
├─ db/            # сессия AsyncSession и URL-хелперы
├─ realtime/      # WebSocket-чат и Redis Pub/Sub
└─ scripts/       # утилиты: seed, export OpenAPI
```

### 3.2 Dependency Injection как механизм связывания слоёв

FastAPI предоставляет встроенный механизм Dependency Injection через Depends. Каждый эндпоинт получает нужный сервис через зависимость-фабрику, а та – сессию базы данных. Схема DI для слоя аутентификации приведена в листинге 1.

Листинг 1 – Dependency Injection для сервиса аутентификации

```
# app/api/dependencies.py
def get_auth_service(
    db: AsyncSession = Depends(db_session.get_db),
) -> AuthService:
    return AuthService(db)

async def get_current_user(
    token: TokenContext = Depends(get_current_user_id),
    db: AsyncSession = Depends(db_session.get_db),
) -> User:
    user = await UserRepository(db).get_by_id(token.user_id)
    if not user:
        raise HTTPException(status_code=404, detail="User not found")
    if token.role != user.role:
        raise HTTPException(
            status_code=401,
            detail={"code": "role_changed", "message": "Re-authenticate"},
        )
    return user
```

Такая схема обеспечивает слабую связанность: в тестах любая зависимость заменяется через `app.dependency_overrides` без изменения кода обработчика. Для ролевых проверок (`get_current_teacher`, `get_current_student`) DI-цепочка расширяется дополнительными зависимостями, исключая дублирование проверок в каждом эндпоинте.

### 3.3 Обработка ошибок

В архитектуре принят следующий подход к ошибкам:

- **Lookup-методы** сервисов (поиск по `id` и т.д.) возвращают `None` при отсутствии сущности. Маршрутизатор проверяет результат и явно поднимает `HTTPException`.

- **Команды** сервисов (создание, обновление, оркестрация с несколькими проверками) поднимают `ServiceError(message, code=...)` –



единственное исключение сервисного слоя. Код ошибки несёт семантику, а HTTP-статус определяет обработчик эндпоинта.

– Неожиданные исключения, не пойманные явно, приводят к ответу 500 Internal Server Error – индикатору ошибки в коде сервера.

В модуле `app/api/http_errors.py` определены фабрики `not_found()`, `forbidden()`, `bad_request()` и `from_service_error()`, которые строят `HTTPException` с унифицированным телом ответа `{"error": {"code", "message"}, "request_id": ...}`. Глобальный обработчик (`app/api/errors.py`) оборачивает любой `HTTPException` в тот же конверт – клиент получает единообразный формат ошибок независимо от места их возникновения.

### **3.4 Диаграмма компонентов**

На рисунке 2 представлена компонентная модель серверной части системы. Диаграмма отражает распределение ответственности между основными подсистемами и показывает, каким образом обеспечивается изоляция бизнес-логики от инфраструктурных деталей.

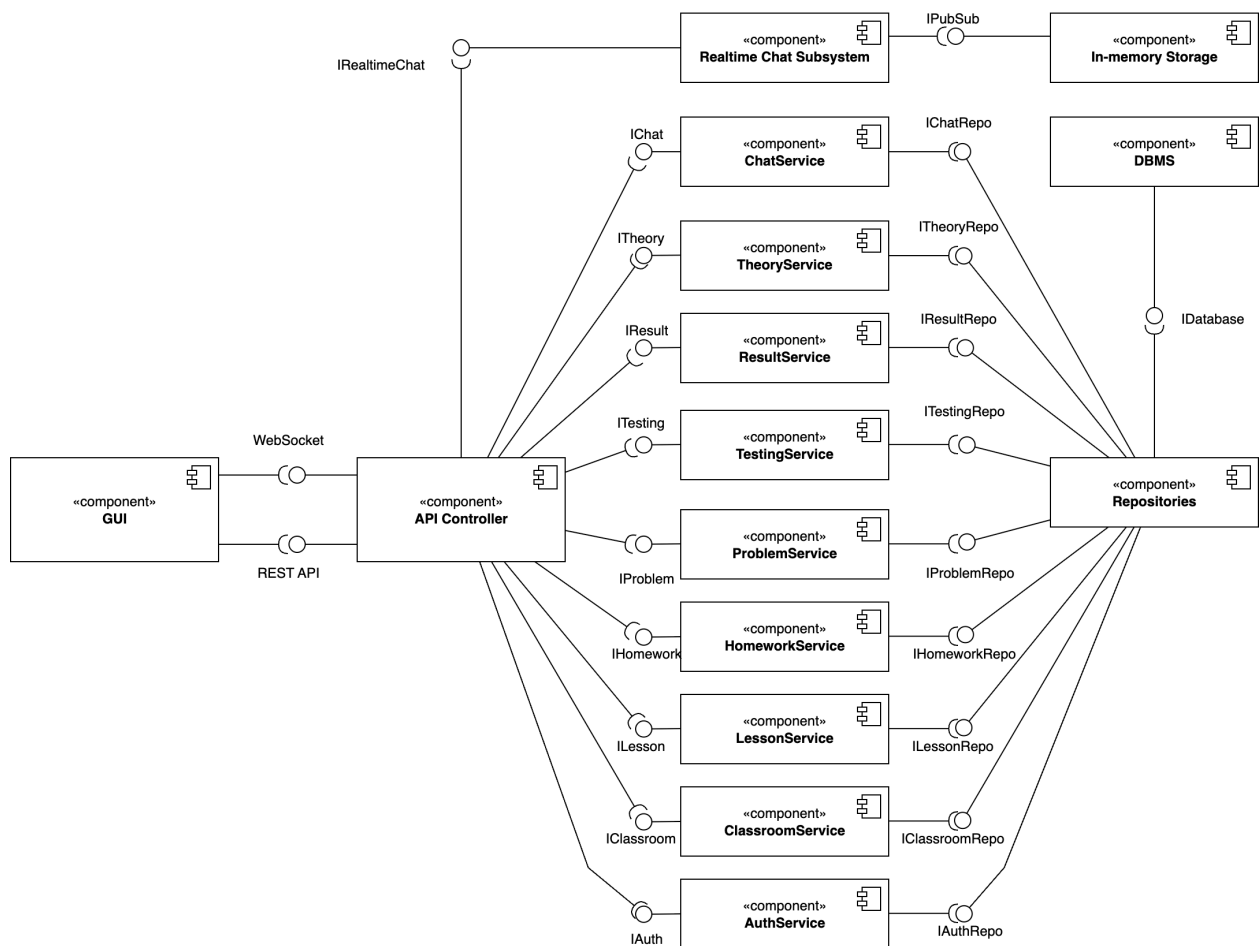


Рисунок 2 – Диаграмма компонентов системы, отражающая Clean Architecture

Центральное место в диаграмме занимает сервисный слой, в котором реализованы основные сценарии использования системы: регистрация и аутентификация пользователей, управление учебными классами, уроками и домашними заданиями, работа с базой задач, автоматическая проверка ответов, расчёт прогресса и предоставление теоретических материалов. Выделение данных сценариев в отдельные сервисы позволяет сосредоточить прикладную логику в одном месте и исключить её дублирование в обработчиках HTTP-запросов.

Сервисный слой взаимодействует с репозиториями, отвечающими за доступ к данным. Такое разделение необходимо для того, чтобы прикладная логика оперировала сущностями предметной области, не включая в себя детали SQL-запросов, транзакций и ORM-моделей. Благодаря этому обеспечивается независимость прикладных правил от конкретной схемы хранения данных и повышается тестируемость системы.

Таким образом, компонентная диаграмма демонстрирует, что серверная часть построена как согласованная система взаимосвязанных подсистем, в которой каждый компонент выполняет строго определённую функцию: приём запроса, реализацию прикладного сценария, доступ к данным или обмен сообщениями в реальном времени. Такое распределение ответственности соответствует принципам Clean Architecture и обеспечивает сопровождаемость разрабатываемого программного обеспечения.

Спроектирована реляционная база данных, охватывающая все ключевые сущности предметной области. ER-диаграмма представлена на рисунке 3.



Основные сущности:

- **users** – профиль пользователя (логин, email, ФИО, активная роль).
- **login\_data** – хеш пароля, изолированный от профиля для разграничения доступа.
- **teachers / students** – профили ролей; один пользователь может иметь оба, переключая активную роль.
- **classrooms** – учебный класс с кодом приглашения.
- **student\_classroom** – членство студента в классе («многие ко многим»).
- **lessons** – урок, привязанный к классу.
- **homework** – домашнее задание, привязанное к уроку.
- **problems** – независимая база задач, переиспользуемая в разных домашних заданиях.
- **homework\_problem** – связь задания и задачи с атрибутом «баллы».
- **statistics** – результат выполнения задания студентом.
- **theme / theory** – иерархия тем и теоретические материалы.
- **chat / message** – чат класса и его сообщения.

Пример ORM-модели пользователя приведён в листинге 2. Принципиальная особенность схемы – отделение `login_data` от `users`: это изолирует хеш пароля от обычных выборок профиля.

## Листинг 2 – ORM-модель пользователя (SQLAlchemy)

```
# app/models/users.py
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    username = Column(String(100), unique=True, nullable=False, index=True)
    email = Column(String(255), unique=True, nullable=False, index=True)
    first_name = Column(String(100), nullable=False)
    last_name = Column(String(100), nullable=False)
    full_name = Column(String(255), nullable=False)
    role = Column(String(20), nullable=False, default="student")
    is_active = Column(Boolean, default=True, nullable=False)
    created_at = Column(DateTime(timezone=True), default=utc_now,
nullable=False)

    login_data = relationship("LoginData", uselist=False, cascade="all, delete-
orphan")
    teacher = relationship("Teacher", uselist=False, cascade="all, delete-
orphan")
    student = relationship("Student", uselist=False, cascade="all, delete-
orphan")
```

Управление миграциями выполняется через Alembic: каждое изменение схемы оформляется в виде пронумерованной ревизии [20].

### 3.6 Результаты проектирования

Разработана детальная архитектура серверной части на основе Clean Architecture. Описаны четыре слоя и их зоны ответственности, показана структура директорий. Обоснован подход к обработке ошибок: сервисы возвращают None для lookup-операций и поднимают ServiceError для команд; HTTP-семантику определяют маршрутизаторы. Описан механизм Dependency Injection, обеспечивающий слабую связанность и тестируемость. Спроектирована реляционная схема базы данных, охватывающая все ключевые сущности предметной области. Разработанная архитектура является основой для реализации серверной части в следующей главе.

## 4 Разработка серверной части интернет-ресурса

### 4.1 Реализация REST API на FastAPI

Серверная часть реализована на Python (версия 3.12+) с использованием FastAPI [15,16]. Все маршруты сгруппированы по функциональным областям и зарегистрированы единым корневым роутером в `app/api/v1/__init__.py`:

- `/api/v1/auth/` – аутентификация и управление ролями
- `/api/v1/classrooms/` – классы, приглашения, чат, WebSocket
- `/api/v1/lessons/` – уроки
- `/api/v1/homework/` – домашние задания
- `/api/v1/problems/` – база задач
- `/api/v1/testing/` – приём ответов, автопроверка
- `/api/v1/statistics/` – прогресс и статистика
- `/api/v1/theory/` – теоретические материалы

При проектировании API соблюдаются стандартные принципы REST [24]. Взаимодействие строится вокруг ресурсов, доступ к которым осуществляется через стандартные HTTP-методы (GET, POST, PATCH, DELETE). Принцип взаимодействия между клиентом и сервером посредством REST API проиллюстрирован на рисунке 4.

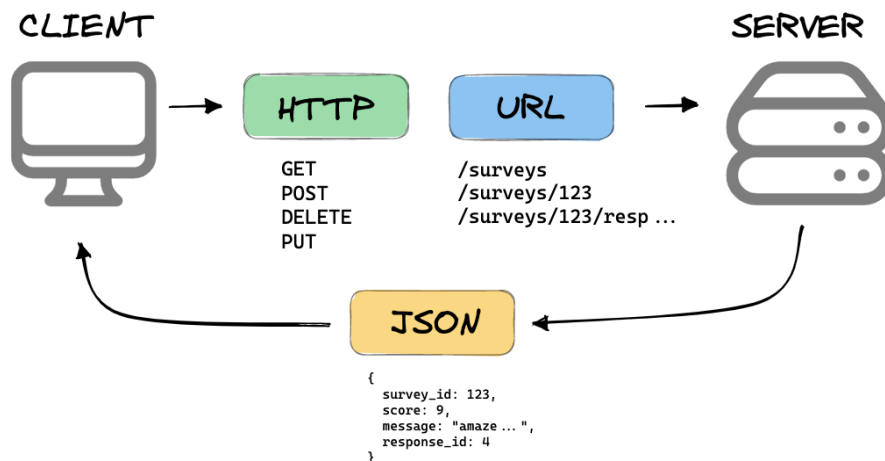


Рисунок 4 – Принцип взаимодействия через REST API

Каждый обработчик выполняет единственную задачу: валидирует входные данные через Pydantic-схему, вызывает сервис и транслирует результат в HTTP-ответ. Пример эндпоинта аутентификации приведён в листинге 3.

### Листинг 3 – Реализация эндпоинта аутентификации

```
# app/api/v1/auth.py
@router.post("/login", response_model=TokenResponse)
async def login(
    credentials: LoginRequest,
    auth_service: AuthService = Depends(deps.get_auth_service),
) -> TokenResponse:
    try:
        token = await auth_service.authenticate(credentials)
    except ServiceError as exc:
        if exc.code == "inactive_user":
            raise http_errors.forbidden(exc.message, code=exc.code)
        raise
    if token is None:
        raise http_errors.unauthorized("Invalid username/email or password")
    return token
```

Листинг демонстрирует принцип явной обработки ошибок: обработчик знает семантику каждого кода и выбирает HTTP-статус самостоятельно. Незнакомое исключение не перехватывается – ответ 500 сигнализирует об ошибке на стороне сервера.

Практическая ценность такого подхода заключается в том, что поведение API становится полностью предсказуемым как для разработчика серверной части, так и для разработчика клиента. Для каждой типовой ситуации – неверные учётные данные, деактивированная учётная запись, отсутствие ресурса или нарушение ограничений доступа – обработчик формирует понятный и однотипный ответ. Это упрощает сопровождение системы и исключает скрытые преобразования ошибок, затрудняющие отладку.

#### 4.2 Реализация сервисного слоя

Сервисный слой реализует бизнес-сценарии и не зависит от HTTP-контекста. Сервисы используют репозитории для доступа к данным, а все проверки прикладных ограничений выполняются до передачи управления на уровень ORM. Такой подход обеспечивает воспроизводимость бизнес-правил независимо от того, каким именно способом был инициирован вызов: через REST API, WebSocket или внутренний скрипт. Ключевой фрагмент сервиса аутентификации приведён в листинге 4.

#### Листинг 4 – Ключевой фрагмент сервиса аутентификации

```
# app/services/auth.py – метод authenticate
async def authenticate(
    self, login: LoginRequest,
) -> TokenResponse | None:
    user = await self.user_repo.get_by_username_or_email(
        login.username_or_email
    )
    if not user:
        return None
    if not user.is_active:
        raise ServiceError("User account is inactive", code="inactive_user")

    info = await self.login_repo.get_by_user_id(user.id)
    if not info or not security.verify_password(
        login.password, info.hashed_password,
    ):
        return None

    token = security.create_access_token(
        {"sub": str(user.id), "role": user.role}
    )
    return TokenResponse(
        access_token=token,
        user=UserResponse.model_validate(user),
    )
```

Сервис `TestingService` реализует ключевой сценарий автоматической проверки ответов студента. Для системы управления обучением данный сервис имеет принципиальное значение, поскольку именно он обеспечивает прикладную ценность платформы: ответ студента не просто сохраняется, а сразу сравнивается с эталоном, после чего обновляются накопленный балл, число попыток и затраченное время. Код соответствующего фрагмента приведён в листинге 5.



## Листинг 5 – Фрагмент реализации автоматической проверки ответа

```
# app/services/testing.py – фрагмент метода submit_answer
async def submit_answer(
    self, answer_data: AnswerSubmit, student_user_id: int,
) -> StatisticsResponse:
    # ... [Проверка прав доступа, существования задачи и её принадлежности к
    ДЗ] ...

    stats = await self.stats_repo.get_or_create_stats(
        student.id, answer_data.homework_id, homework.max_score,
    )

    # Автоматическая проверка ответа студента
    is_correct = self._check_answer(answer_data.answer, problem.correct_answer)

    # Начисление баллов с ограничением по максимальному баллу
    new_score = (
        min(stats.score + hw_problem.points, stats.max_score) if is_correct
        else stats.score
    )

    # Фиксация попытки и затраченного времени
    updated = await self.stats_repo.update(stats.id, {
        "score": new_score, "status": "in_progress",
        "attempts_count": stats.attempts_count + 1,
        "time_spent_minutes": stats.time_spent_minutes +
answer_data.time_spent_minutes,
    })
    return StatisticsResponse.model_validate(updated)

def _check_answer(self, student: str, correct: str | None) -> bool:
    if not correct: return False
    return student.strip().lower() == correct.strip().lower()
```

### 4.3 Слой безопасности

Безопасность реализована на двух уровнях: хеширование паролей и аутентификация через JWT.

Пароли хешируются алгоритмом Argon2id – устойчивым к атакам с использованием GPU и ASIC, рекомендованным для хранения учётных данных [25]. JWT-токены формируются с кратким сроком действия и содержат

идентификатор пользователя и активную роль, что позволяет обнаруживать смену роли без обращения к базе данных.

Выбор Argon2id обусловлен необходимостью защищённого хранения паролей в условиях современного уровня вычислительных угроз. В отличие от простых криптографических хеш-функций, данный алгоритм учитывает не только вычислительную сложность, но и объём используемой памяти, что повышает стоимость атак перебором. Применение JWT, в свою очередь, позволяет реализовать безсессионную аутентификацию и уменьшить нагрузку на сервер при каждом запросе, сохраняя возможность централизованной проверки роли и срока действия токена.

Листинг 6 – Реализация безопасности: Argon2id и JWT

```
# app/core/security.py
_MEMORY_COST_KIB = 1024 * 64 # 64 MB

ph = argon2.PasswordHasher(
    time_cost=2, memory_cost=_MEMORY_COST_KIB,
    parallelism=1, hash_len=32, salt_len=16,
)

def verify_password(plain: str, hashed: str) -> bool:
    try:
        ph.verify(hashed, plain)
        return True
    except argon2.exceptions.VerifyMismatchError:
        return False

def create_access_token(data: dict) -> str:
    payload = {**data, "exp": utc_now() +
timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)}
    return jose_jwt.encode(payload, settings.SECRET_KEY,
algorithm=settings.ALGORITHM)
```

#### 4.4 Управление конфигурацией и запуск

Конфигурация приложения хранится в переменных окружения, загружаемых через pydantic-settings [26]. Такой способ конфигурирования обеспечивает независимость параметров запуска от исходного кода и соответствует общепринятой практике построения серверных приложений. Точка входа (app/main.py) подключает middleware (CORS,

RequestIdMiddleware), глобальные обработчики ошибок и API-роутер с префиксом `/api/v1`. При наличии `REDIS_URL` в момент старта инициализируется Redis Pub/Sub-брокер для многоэкземплярного WebSocket-чата; без него приложение стартует без realtime-компонента.

Серверная часть автоматически формирует OpenAPI-спецификацию, которая используется как машинно-читаемое описание контракта API. Это важно не только для ручной проверки эндпоинтов, но и для интеграции с клиентской частью, поскольку OpenAPI-описание может выступать основой для кодогенерации, тестирования и валидации совместимости версий интерфейса. На рисунке 5 показан интерфейс OpenAPI-документации.

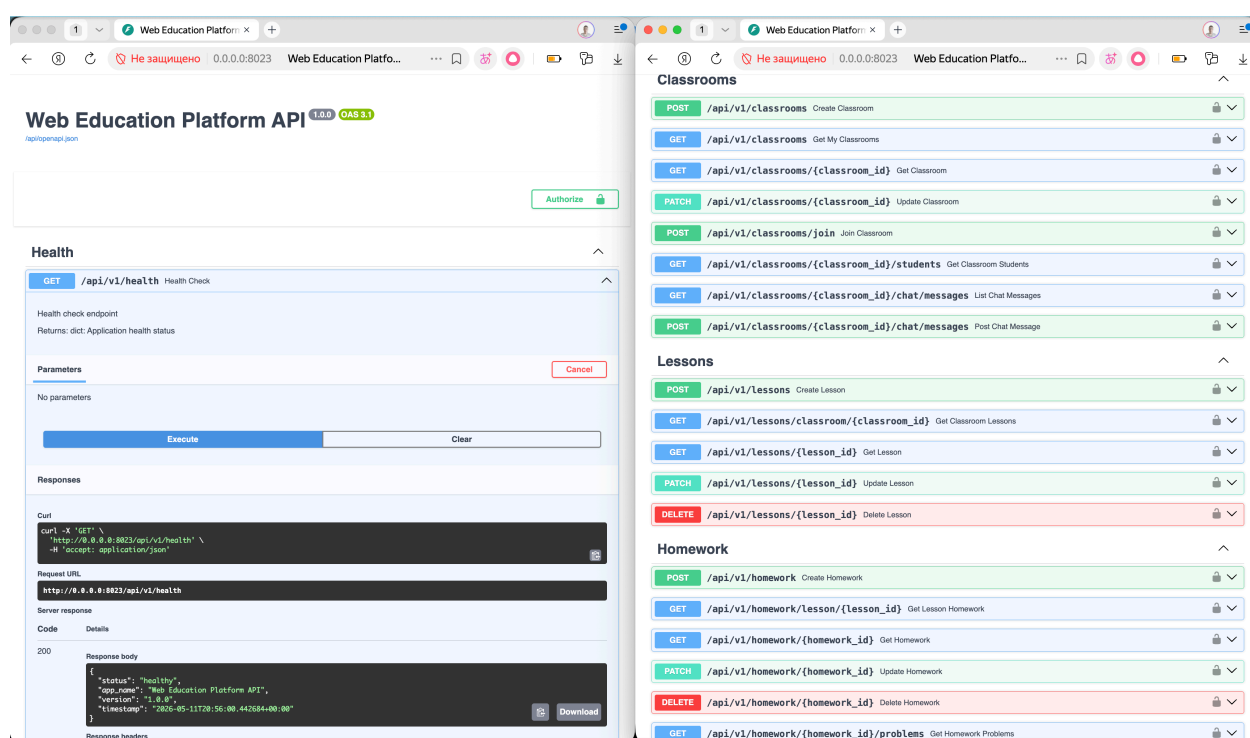


Рисунок 5 – Интерфейс автоматически сформированной OpenAPI-документации

## 4.5 Тестирование

Принципы Clean Architecture обеспечили практическую изоляцию сервисного слоя от веб-слоя. Тесты используют сервисы напрямую, без поднятия FastAPI. Для каждого теста создаётся отдельная схема PostgreSQL, которая накатывает таблицы из метаданных SQLAlchemy и удаляется по завершении. Это позволяет проверять бизнес-логику в реалистичных условиях при полной изоляции между тестами [19]. В составе проекта реализованы

тесты для AuthService, TestingService, ChatService, ProblemService, ResultService, TheoryService, а также для модулей realtime.

Автоматизированное тестирование выполняется с помощью pytest и служит средством проверки корректности реализованных сценариев. Тесты охватывают регистрацию и аутентификацию, автоматическую проверку ответов, подсистему обмена сообщениями, OpenAPI-контракт, расчёт прогресса и работу realtime-компонентов. На рисунке 6 показан результат выполнения набора тестов серверной части.

```
> pytest tests/ -v --tb=short --color=yes -ra

===== test session starts =====
platform darwin -- Python 3.13.0, pytest-8.4.2, pluggy-1.6.0 -- /Users/kerrodar/Dev/wep-education-platform/backend/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /Users/kerrodar/Dev/wep-education-platform/backend
configfile: pyproject.toml
plugins: anyio-4.12.0
collected 30 items

tests/test_auth_service.py::test_register_user_teacher[asyncio] PASSED [ 3%]
tests/test_auth_service.py::test_register_user_student[asyncio] PASSED [ 6%]
tests/test_auth_service.py::test_switch_role_creates_profile_and_updates_active_role[asyncio] PASSED [ 10%]
tests/test_auth_service.py::test_old_token_becomes_invalid_after_role_switch[asyncio] PASSED [ 13%]
tests/test_auth_service.py::test_register_duplicate_email[asyncio] PASSED [ 16%]
tests/test_auth_service.py::test_authenticate_success[asyncio] PASSED [ 20%]
tests/test_auth_service.py::test_authenticate_wrong_password[asyncio] PASSED [ 23%]
tests/test_auth_service.py::test_authenticate_nonexistent_user[asyncio] PASSED [ 26%]
tests/test_chat_pagination.py::test_message_repo_tail_and_before_id[asyncio] PASSED [ 30%]
tests/test_chat_service.py::test_chat_service_teacher_can_post_and_list[asyncio] PASSED [ 33%]
tests/test_chat_service.py::test_chat_service_rejects_non_member_student[asyncio] PASSED [ 36%]
tests/test_openapi_contract.py::test_openapi_contains_error_response_schema PASSED [ 40%]
tests/test_openapi_contract.py::test_openapi_operations_have_standard_error_responses PASSED [ 43%]
tests/test_openapi_contract.py::test_openapi_progress_endpoints_are_typed PASSED [ 46%]
tests/test_problem_service.py::test_create_problem[asyncio] PASSED [ 50%]
tests/test_problem_service.py::test_get_all_problems[asyncio] PASSED [ 53%]
tests/test_problem_service.py::test_get_problem_by_id[asyncio] PASSED [ 56%]
tests/test_problem_service.py::test_get_nonexistent_problem[asyncio] PASSED [ 60%]
tests/test_problem_service.py::test_update_problem[asyncio] PASSED [ 63%]
tests/test_problem_service.py::test_delete_problem[asyncio] PASSED [ 66%]
tests/test_realtime_auth.py::test_extract_bearer_token PASSED [ 70%]
tests/test_realtime_auth.py::test_extract_websocket_token_prefers_query_param PASSED [ 73%]
tests/test_realtime_auth.py::test_extract_websocket_token_falls_back_to_header PASSED [ 76%]
tests/test_realtime_connection_manager.py::test_connection_manager_connect_broadcast_disconnect[asyncio] PASSED [ 80%]
tests/test_realtime_presence_store.py::test_presence_store_online_filters_stale_members[asyncio] PASSED [ 83%]
tests/test_realtime_presence_store.py::test_presence_store_typing_stops_on_offline[asyncio] PASSED [ 86%]
tests/test_result_service.py::test_result_service_classroom_progress_aggregates[asyncio] PASSED [ 90%]
tests/test_result_service.py::test_result_service_student_progress_aggregates[asyncio] PASSED [ 93%]
tests/test_theory_service.py::test_theory_service_filters_unpublished_for_students[asyncio] PASSED [ 96%]
tests/test_token_role_mismatch.py::test_get_current_user_rejects_role_mismatch[asyncio] PASSED [100%]
```

Рисунок 6 – Автоматизированные тесты серверной части

## 4.6 Клиентское представление

Клиентская часть платформы реализована на React 18 с TypeScript [21]. Архитектура фронтенда построена по методологии Feature-Sliced Design (FSD) [22], которая структурирует код в шесть слоёв по убыванию области ответственности. Принципиальная схема данной методологии представлена на рисунке 7.

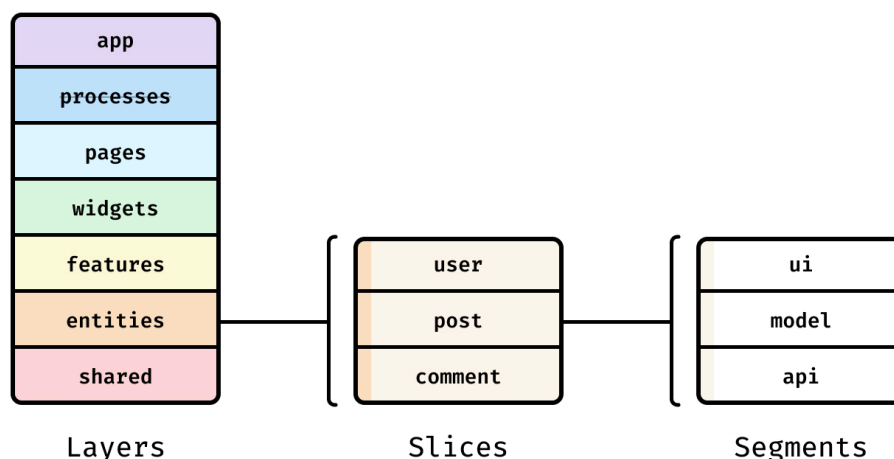


Рисунок 7 – Схема слоёв методологии Feature-Sliced Design

В рамках проекта слои распределены следующим образом:

```
src/
├─ app/      – инициализация, провайдеры, роутинг
├─ pages/    – страницы (Login, Classrooms, Lesson, ...)
├─ widgets/  – составные блоки (Header, Sidebar, ChatPanel)
├─ features/ – сценарии пользователя (JoinClassroom,
SubmitAnswer, ...)
├─ entities/ – доменные данные (Classroom, Lesson, User, Message)
└─ shared/   – переиспользуемые утилиты, UI-kit, HTTP-клиент
```

Взаимодействие с сервером ведётся через axios-клиент в `shared/api/axios.ts`. Серверный контракт описан в файле `openapi.json`, генерируемом бэкенд-командой `python -m app.scripts.export_openapi`. Управление серверным состоянием реализовано через TanStack Query v4, локальное состояние – через Zustand. Формы валидируются с помощью react-hook-form и zod. UI построен на примитивах Radix UI и стилизован через Tailwind CSS.

Требование межстраничной навигации реализуется за счёт маршрутизации в слое `app`, где определяются основные пользовательские переходы между экранами входа, списком классов, страницами уроков, домашними заданиями и представлением результатов. Такое построение навигации позволяет сохранить целостность пользовательского сценария: преподаватель переходит от списка учебных классов к конкретному уроку и связанным с ним заданиям, а студент – от структуры курса к выполнению конкретного домашнего задания и просмотру результата проверки.

Требование современного веб-дизайна обеспечивается использованием единой компонентной базы и общей системы стилизации. Применение Radix UI и Tailwind CSS позволяет поддерживать единообразие визуальных элементов, адаптивность интерфейса и предсказуемость поведения экранов, что особенно важно для образовательной платформы, ориентированной на ежедневное использование преподавателями и студентами.

Подобная организация клиентской части важна с точки зрения backend-разработки, поскольку позволяет выстроить устойчивый канал взаимодействия между сервером и пользовательским интерфейсом. Чёткое разделение страниц, сущностей и пользовательских сценариев снижает риск рассогласования между моделью данных на сервере и представлением этих данных на клиенте. Наличие единого OpenAPI-контракта дополнительно упрощает синхронизацию обеих частей системы.

Интерфейс страницы класса преподавателя должен решать две практические задачи: быстро показать состав учебной группы и дать доступ к операциям управления ими. На рисунке 8 показан интерфейс, в котором преподаватель может видеть список учеников, их успеваемость, код приглашения, и элементы навигации, что позволяет быстро ориентироваться в текущем состоянии учебного процесса.

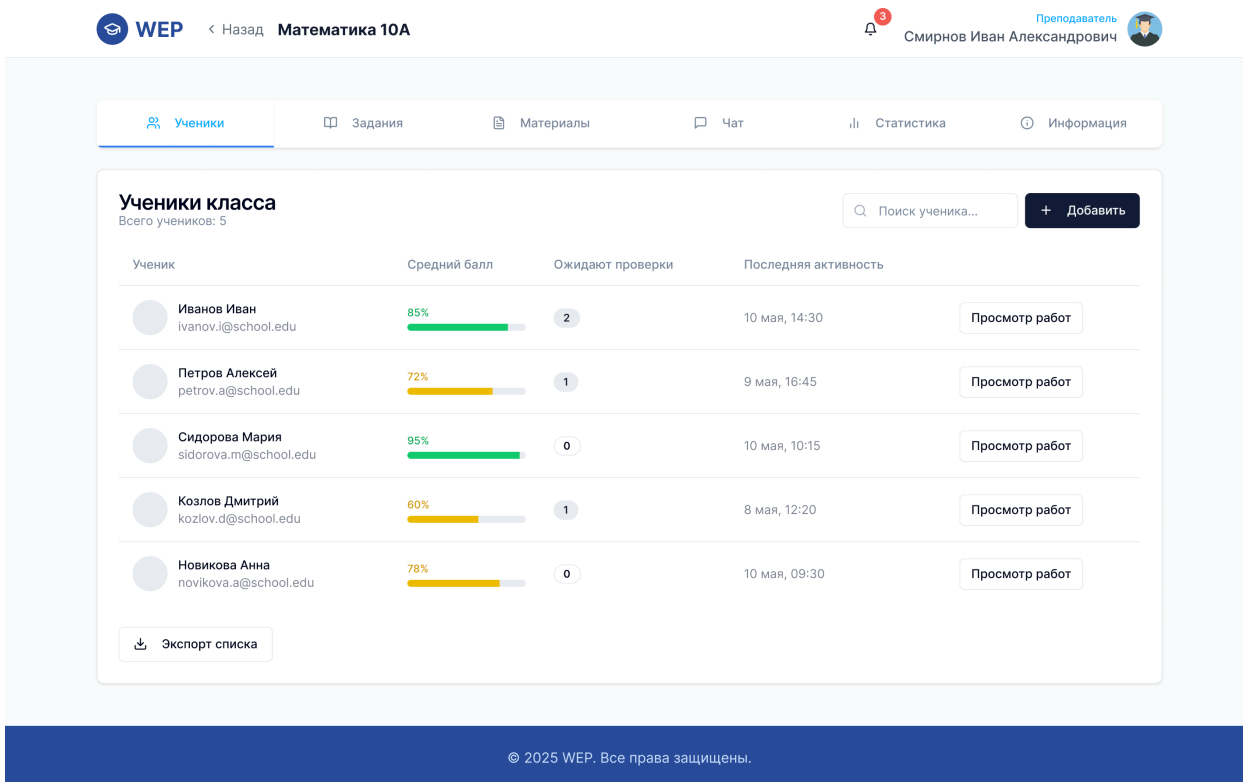


Рисунок 8 – Главная страница класса преподавателя

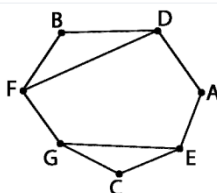
Страница выполнения домашнего задания для студента должна концентрировать навигацию по заданиям, отображать подробное условие и предоставлять удобную возможность для ввода ответа. На рисунке 9 показан интерфейс, в котором студент может перемещаться между заданиями и вводить ответы.

Открыть описание домашнего задания ▾

1 2 3 4 5

Найдите корни уравнения:  $x^2 - 4 = 0$  и заполните таблицу соответствия между пунктами.

| Номер пункта | Номер пункта |   |   |   |   |   |   |
|--------------|--------------|---|---|---|---|---|---|
|              | 1            | 2 | 3 | 4 | 5 | 6 | 7 |
| 1            |              | * | * | * | * |   |   |
| 2            | *            |   | * |   |   | * |   |
| 3            | *            | * |   |   |   |   |   |
| 4            | *            |   |   | * | * |   | * |
| 5            |              |   |   | * |   | * | * |
| 6            |              | * |   | * | * |   |   |
| 7            |              |   |   | * | * |   |   |



Ваш ответ

Вложения:

Нет вложений

Прикрепить файл

Сохранить ответ

Рисунок 9 – Страница выполнения домашнего задания для студента

Связь фронтенда с бэкендом реализована через единый API-контракт OpenAPI: изменение серверного эндпоинта отражается в спецификации и может быть подхвачено кодогенерацией на стороне клиента, что обеспечивает согласованность интерфейсов без ручной синхронизации.

#### 4.7 Итоги разработки

Реализована серверная часть на Python с использованием FastAPI, PostgreSQL и SQLAlchemy. REST API включает девять групп эндпоинтов, покрывающих все функциональные требования системы. Обработка ошибок построена явно: сервисы возвращают None для lookup-операций и поднимают ServiceError для командных сценариев; HTTP-статус выбирает маршрутизатор. Обеспечена безопасность: Argon2id для хранения паролей и JWT для аутентификации. Тестируемость архитектуры подтверждена набором автоматизированных тестов, работающих на уровне сервисного слоя без HTTP. Реализована клиентская часть на React + TypeScript по методологии Feature-Sliced Design, потребляющая REST API через OpenAPI-контракт.



## ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы разработана серверная часть веб-приложения для управления образовательными курсами на основе архитектурного паттерна Clean Architecture.

На первом этапе выполнен анализ предметной области систем управления обучением. Сформированы функциональные и нефункциональные требования к серверной части. Проведён обзор четырёх LMS-платформ – Moodle, Google Classroom, Stepik и Фоксфорд – и построена матрица отслеживаемости, выявившая ниши, в которых разрабатываемая система обеспечивает функциональные преимущества: независимая переиспользуемая база задач, встроенный realtime-чат, полноценный REST API с возможностью развёртывания на собственном сервере.

На втором этапе проведён сравнительный анализ архитектурных паттернов DDD, Clean Architecture и MVC/MVT. Показано, что для разрабатываемой системы наиболее целесообразна Clean Architecture, поскольку она обеспечивает явное разделение ответственности, высокую тестируемость и независимость прикладной логики от инфраструктурных компонентов. Сформирован технологический стек: Python версии 3.12+, FastAPI, PostgreSQL, SQLAlchemy.

На третьем этапе разработана детальная архитектура серверной части. Описаны слои API, Service, Repository и Models, механизм Dependency Injection и принятый подход к обработке ошибок. Спроектирована реляционная схема базы данных, охватывающая все сущности предметной области.

На четвёртом этапе реализованы серверная и клиентская части приложения. Серверная часть включает девять групп REST-эндпоинтов, сервисный слой с автоматической проверкой ответов, слой безопасности (Argon2id, JWT) и WebSocket-чат с опциональным Redis Pub/Sub-брокером. Клиентская часть реализована на React 18 + TypeScript по методологии Feature-Sliced Design. Корректность архитектурных решений подтверждена набором автоматизированных тестов, работающих на уровне сервисного слоя без HTTP-слоя.

Полученные результаты подтверждают применимость Clean Architecture и выбранного технологического стека для построения серверной части системы управления обучением. Разработанная архитектура допускает дальнейшее расширение функциональности и масштабирование без переписывания ядра системы.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Минакова О.В. Программная инженерия: Основные принципы, методы и инструменты [Электронный ресурс]: учебник для вузов ISBN 978–5–507–49278–7. Санкт-Петербург: Лань, 2024. 212 с. URL: <https://e.lanbook.com/book/414989> (дата обращения: 11.05.2026).
2. Rosário A.T., Dias J.C. Learning Management Systems in Education: Research and Challenges: DOI: 10.4018/978-1-6684-4706-2.ch003 // Digital Active Methodologies for Educative Learning Management / под ред. Geada N., Jamil G.L. Hershey, PA: IGI Global, 2022. Сс. 47–77.
3. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения: ISBN 978–5–4461–1903–1. 2-е изд. Санкт-Петербург: Питер, 2021. 352 с.
4. Киселева Т.В. Программная инженерия [Электронный ресурс]: учебное пособие. Ставрополь: СКФУ, 2017. Т. 2. 100 с. URL: <https://e.lanbook.com/book/155149> (дата обращения: 11.05.2026).
5. Туманова М.Б., Михайлова Е.К., Муравьева Е.А. Проектирование программных систем [Электронный ресурс]: учебное пособие ISBN 978–5–7339–2050–4. Москва: РТУ МИРЭА, 2023. 138 с. URL: <https://e.lanbook.com/book/398273> (дата обращения: 11.05.2026).
6. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. Москва: Стандартинформ, 2017.
7. Moodle — Open-source learning platform [Электронный ресурс]. URL: <https://moodle.org/> (дата обращения: 11.05.2026).
8. Google Classroom — официальный сайт [Электронный ресурс]. URL: <https://edu.google.com/intl/ru/workspace-for-education/classroom/> (дата обращения: 11.05.2026).
9. Stepik — онлайн-платформа для обучения [Электронный ресурс]. URL: <https://stepik.org/> (дата обращения: 11.05.2026).
10. Фоксфорд — онлайн-школа для учеников 1–11 классов [Электронный ресурс]. URL: <https://foxford.ru/> (дата обращения: 11.05.2026).

11. Персиваль Г., Грегори Б. Паттерны разработки на Python: TDD, DDD и событийно-ориентированная архитектура: ISBN 978–5–4461–1969–7. Санкт-Петербург: Питер, 2022. 336 с.
12. Python 3 Documentation [Электронный ресурс]. URL: <https://docs.python.org/3/> (дата обращения: 11.05.2026).
13. Баланов А.Н. Комплексное руководство по разработке: от мобильных приложений до веб-технологий [Электронный ресурс]: учебное пособие для вузов ISBN 978–5–507–53193–6. 2-е изд. Санкт-Петербург: Лань, 2025. 412 с. URL: <https://e.lanbook.com/book/478178> (дата обращения: 11.05.2026).
14. Stack Overflow Developer Survey 2025 [Электронный ресурс]. 2025. URL: <https://survey.stackoverflow.co/2025/> (дата обращения: 11.05.2026).
15. Елисеев А.И., Минин Ю.В. Разработка программных интерфейсов веб-приложений с использованием фреймворка FastAPI [Электронный ресурс]: учебное пособие ISBN 978–5–8265–2821–1. Тамбов: ТГТУ, 2024. 81 с. URL: <https://e.lanbook.com/book/472310> (дата обращения: 11.05.2026).
16. Ramírez S. FastAPI – официальная документация [Электронный ресурс]. URL: <https://fastapi.tiangolo.com/> (дата обращения: 11.05.2026).
17. PostgreSQL Documentation [Электронный ресурс]. URL: <https://www.postgresql.org/docs/> (дата обращения: 11.05.2026).
18. Волк В.К., Осеев В.Ю., Черепанов О.С. Базы данных [Электронный ресурс]: учебник ISBN 978–5–9729–2594–0. Вологда: Инфра-Инженерия, 2025. 544 с. URL: <https://e.lanbook.com/book/499772> (дата обращения: 11.05.2026).
19. SQLAlchemy 2.0 Documentation [Электронный ресурс]. URL: <https://docs.sqlalchemy.org/en/20/> (дата обращения: 11.05.2026).
20. Alembic – официальная документация [Электронный ресурс]. URL: <https://alembic.sqlalchemy.org/> (дата обращения: 11.05.2026).
21. React Documentation [Электронный ресурс]. URL: <https://react.dev/> (дата обращения: 11.05.2026).

22. Feature-Sliced Design: официальная документация [Электронный ресурс]. URL: <https://feature-sliced.github.io/documentation/ru/> (дата обращения: 11.05.2026).
23. Водяхо А.И. и др. Архитектурные решения информационных систем [Электронный ресурс]: учебник для вузов ISBN 978–5–507–44710–7. 3-е изд. Санкт-Петербург: Лань, 2022. 356 с. URL: <https://e.lanbook.com/book/254624> (дата обращения: 11.05.2026).
24. Алпатов А.Н. Интерфейсы прикладного программирования [Электронный ресурс]: учебное пособие ISBN 978–5–7339–2342–0. Москва: РТУ МИРЭА, 2024. 157 с. URL: <https://e.lanbook.com/book/457043> (дата обращения: 11.05.2026).
25. Хоффман Э. Безопасность веб-приложений: ISBN 978–5–4461–1706–8. Санкт-Петербург: Питер, 2021. 336 с.
26. Pydantic V2 Documentation [Электронный ресурс]. URL: <https://docs.pydantic.dev/latest/> (дата обращения: 11.05.2026).